

Appendix P

Hints for programming

The official rules for writing a program are that it should involve three persons. One for the design, one for the implementation, and one for the testing of it. Most probably you are doing all steps on your own! The reason why there is a need for three persons is that if you do not realize a certain aspect in the design phase, it is very likely that you do not think about this either while implementing or testing a program. The following tips may be useful in developing a program:

- If you are modifying a program, first try to understand every line of the starting code. If you do not understand the starting point, it is very dangerous to make modifications.
- Try to develop a modification plan consisting of the following steps:
 1. Make a copy of the program in a new directory.
 2. Describe in words why and how a certain part of the code needs to be modified.
 3. Start the programming by writing comment lines describing what the modifications are and make the modifications according to your written instructions. If you find during the implementations that the original ideas were not correct, describe this in your notes. This is to ensure that you think before you do. This may sound obvious and it is, but as you may find out it is a very difficult rule to stick to.
 4. Do not start modifying the entire program. Try to do it in steps. Try to test the modifications as soon as possible. For example, if the program is a Gibbs-ensemble simulation you probably have to modify the parts of the code that perform particle displacements, volume changes, and particle exchanges. However, there is no need to do all this at the same time. First modify the particle displacement and make all the tests you can (for example, energy conservation) without changing the code for the volume displacement and the particle exchange. Only then move to the next part of the code that must be changed.
 5. The first time you run the modified code, you might want to use array-bound checking (this option is not always used by default). Such checks will slow down the code, but are very useful for detecting errors. It is also useful to use code checkers.
 6. Try to find limiting cases for which your program should reproduce known results. For example, if you have written a program for a mixture, the results should be identical to the pure component results if all

interaction parameters are taken to be identical. Sometimes, the results of a calculation can be compared with exact results, e.g., in the limit where the system should behave as an ideal gas. Such a comparison is, of course, not always possible, but if it is possible it provides an important test.

7. For a standard (constant NVE) Molecular Dynamics program, it is a good idea to check the conservation of the total energy. In most systems, the total momentum should also be conserved (usually zero).
 8. In a Monte Carlo program, one has to calculate energy differences for each trial move. It is a good idea to write also some code for computing the total energy of the system by brute force, i.e., without using any sophisticated (time-saving) tricks. Compare the total energy change thus calculated during a simulation, with the change that is computed by adding all the energy changes during individual trial moves: the sum of all the partial energy changes should equal the difference between the initial and final energy computed by brute force to within the error expected on the basis of the precision of multiple additions.
 9. The results of a MC simulation might be dependent on the (quasi) random number generator that is used: there is no such thing as a true random number generator, but some are less bad than others. It is always a good idea to do a MC simulation with two different random number generators. See, for example, ref. [786] for a discussion about this topic.
- If you obtain numbers for the observables that you are computing, try to understand them and ask yourself whether they are reasonable. When using reduced units, most properties are of $\mathcal{O}(1)$, implying that if you find, say, -2.4 the number may be correct, but if you find 2.4×10^{18} , you should be suspicious.
 - If you are writing a parallel code, we strongly recommend that you make a detailed workplan before you start programming. A common mistake is that you forget to pass system variables (for example, temperature or masses of the particles) to all processors.